



# Deadlocks

Universidade Estadual de Mato Grosso do Sul  
Bacharelado em Ciência da Computação  
Sistemas Operacionais  
Prof. Fabrício Sérgio de Paula

# Tópicos

- Modelo de sistema
- Caracterização do deadlock
- Métodos para manipulação de deadlocks
- Prevenção de deadlocks
- Impedimento de deadlocks
- Detecção de deadlocks
- Recuperação de deadlocks

# Modelo de sistema

- Sistema consiste de:
  - Número finito de recursos
    - Espaço de memória, ciclos CPU, arquivos, dispositivos E/S
    - Várias instâncias de recursos (ex.: 2 CPUs, 3 impressoras, etc.)
  - Recursos distribuídos entre processos
  - Processo:
    1. Solicita acesso a instância de tipo de recurso
    2. Caso consiga acesso, utiliza
    3. Após acesso, libera recurso para novo uso

# Modelo de sistema

- Solicitação e liberação de recursos
  - Através de chamadas ao sistema operacional
    - request, release device
    - open, close file
    - allocate, free memory
  - SO armazena informação sobre
    - Recursos livres, alocados
    - Processos utilizando recursos alocados
    - Processos em fila de espera por recursos

# Modelo de sistema

- Processos em deadlock: aguardam liberação de algum tipo de recurso
  - Físicos: dispositivos de armazenamento, espaço em memória, impressora, ciclos de CPU, etc.
  - Lógicos: arquivos, semáforos, etc.
- Aplicações concorrentes/multithreads
  - Necessitam de atenção especial do programador
  - Boas candidatas a deadlocks!

# Caracterização do deadlock

- Condição necessária: situação de deadlock pode surgir se **todas** as seguintes condições ocorrerem ao mesmo tempo:
  1. Exclusão mútua: existe pelo menos um recurso que não pode ser acessado simultaneamente
  2. Posse e espera: processo tem posse de pelo menos um recurso e aguarda por outro que não está livre
  3. Não preempção: só o processo que utiliza pode liberar espontaneamente o recurso
  4. Espera circular: deve existir  $P_0, P_1, \dots, P_n$ , de forma que  $P_0$  espera por  $P_1$ ,  $P_1$  por  $P_2$ , ...,  $P_{n-1}$  por  $P_n$ , e  $P_n$  por  $P_0$

# Caracterização do deadlock

- Grafo de alocação de recursos
  - Grafo orientado que pode descrever deadlock
    - Conjunto de vértices  $V$
    - Conjunto de arestas  $A$
  - $V$  é particionado em
    - $P = \{P_1, P_2, \dots, P_n\}$ : todos os processos ativos no sistema
    - $R = \{R_1, R_2, \dots, R_m\}$ : todos os tipos de recursos no sistema
  - Tipos de arestas
    - Aresta de solicitação: aresta direcionada  $P_i \rightarrow R_j$ 
      - $P_i$  solicitou uma instância do recurso  $R_j$  e está aguardando
    - Aresta de atribuição: aresta direcionada  $R_j \rightarrow P_i$ 
      - Uma instância do recurso  $R_j$  foi alocada ao processo  $P_i$

# Caracterização do deadlock

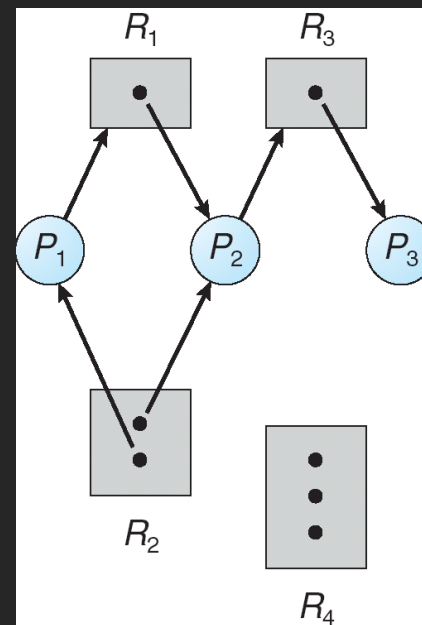
- Ex.: grafo de alocação de recursos

- Instâncias:

- 1 do tipo R1
- 2 do tipo R2
- 1 do tipo R3
- 3 do tipo R4

- Estados dos processos

- P1: posse de 1 instância de R2 e aguardando 1 instância do tipo R1
- P2: posse de 1 instância de R1 e R2 e aguardando 1 instância R3
- P3: posse de 1 instância de R3



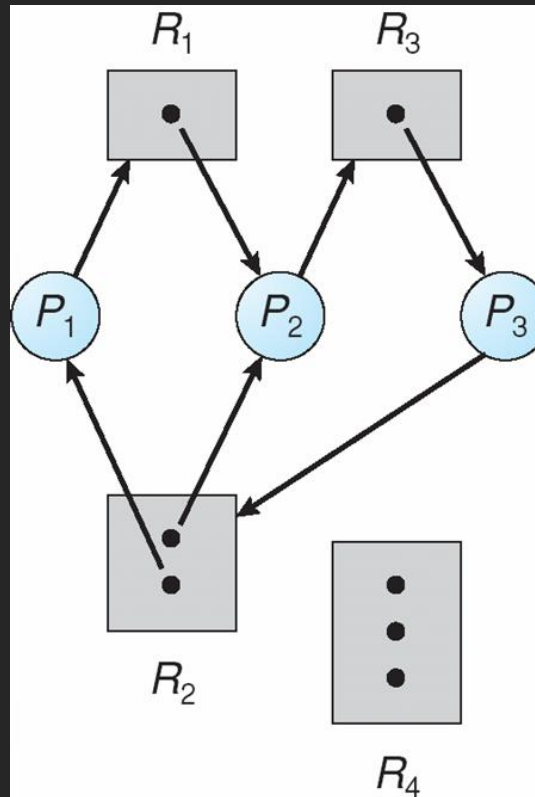


# Caracterização do deadlock

- Grafo de alocação de recursos:
  - Se não contiver ciclos: nenhum processo estará em deadlock
  - Se contiver um ciclo: poderá existir um deadlock
    - Se cada tipo de recurso tiver exatamente 1 instância: ciclo implica em deadlock
    - Se ciclo envolver apenas conjunto de recursos com uma única instância, então há deadlock
    - Cada processo envolvido no ciclo do deadlock está em deadlock

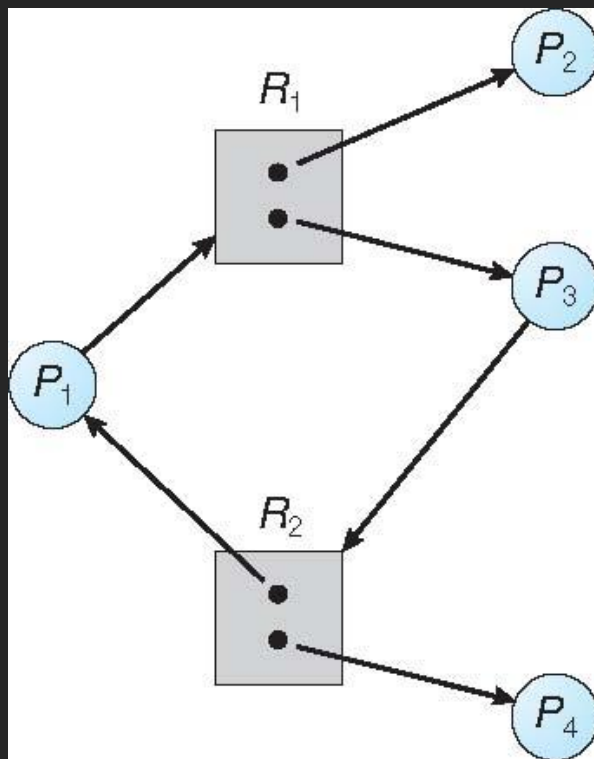
# Caracterização do deadlock

- Ex.: grafo de alocação de recursos com um deadlock entre  $P_1$ ,  $P_2$  e  $P_3$



# Caracterização do deadlock

- Ex.: grafo de alocação de recursos com ciclo, mas sem deadlock



# Métodos para manipulação de deadlocks

- Lidar com deadlocks pode ser feito de 3 maneiras:
  - Usar um protocolo para evitar deadlocks: garante que o sistema jamais entrará em deadlock
  - Permitir o sistema entrar em estado de deadlock
    - Solução: detectar deadlocks e recuperar sistema
  - Ignorar o problema: SO “finge” que deadlocks não ocorrerão
    - Maior parte dos SOs ignora

# Métodos para manipulação de deadlocks

- Prevenir deadlocks: garantir que uma das 4 condições necessárias para o deadlock não venha ocorrer
- Impedir deadlocks: SO analisa e contabiliza informações prévias sobre alocação de recursos

# Prevenção de deadlocks

- A prevenção pode ser feita analisando cada uma das 4 condições necessárias para o deadlock
- Exclusão mútua:
  - Essa condição deve existir para recursos não compartilháveis ao mesmo tempo (ex.: impressora): processos devem esperar
  - Recursos compartilháveis que não exigem exclusão mútua (ex.: arquivos somente para leitura): processos não precisam esperar
  - Em geral: não se pode negar a exclusão mútua
    - Alguns recursos são intrinsecamente não-compartilháveis (memória, arquivos leitura/escrita, etc.)
    - Dessa forma, não há como prevenir deadlocks

# Prevenção de deadlocks

- Posse e espera:
  - Para garantir a não ocorrência, um processo só pode solicitar um recurso se não estiver utilizando outro recurso
- Protocolo de uso possível:
  - Cada processo deve solicitar e receber todos os seus recursos antes de iniciar execução (ou uso)
- Protocolo alternativo:
  - Um processo pode solicitar recursos somente quando não estiver retendo (usando) outros
    - Um processo pode solicitar recursos e utilizá-los
    - Para recursos adicionais, deve liberar todos os que estiverem alocados

# Prevenção de deadlocks

- Posse e espera (cont.):
  - Desvantagens desses protocolos de uso:
    - Baixa utilização de recursos
      - Recursos podem ser alocados por grande período, mesmo quando não utilizados
    - Pode haver inanição
      - Processo que necessite de muitos recursos pode ficar indefinidamente esperando (pelo menos 1 recurso sempre alocado a outro processo)



# Prevenção de deadlocks

- Não preempção:
  - Protocolo: se um processo estiver de posse de recursos e solicitar outro que não puder ser imediatamente alocado
    - Todos os recursos alocados serão preemptados
      - Inicialmente liberados
      - Em seguida, incluídos na lista de recursos que o processo está esperando
      - Processo será retomado quando puder obter todos os recursos necessários para execução

# Prevenção de deadlocks

- Não preempção (cont.):
  - Protocolo alternativo:
    - Se recursos necessários para execução de um processo P estiver com processo Q e Q estiver em espera por outros
      - Retira recursos de Q e aloca-os a P
    - Caso contrário, P precisará aguardar e seus recursos podem ser preemptados
  - Protocolo muito usado para recursos que podem ter seu estado facilmente (rapidamente) salvos e restaurados (ex.: registradores de CPU e espaço de memória)

# Prevenção de deadlocks

- Espera circular:
  - Protocolo para evitar espera circular:
    - Todos os recursos são ordenados
    - Exigência: cada processo só pode solicitar recursos em ordem crescente
    - Exigência alternativa: processo só pode solicitar instância de recurso  $R_j$  qualquer se liberar todos os recursos  $R_i$  que, na ordenação, aparecem depois
  - Essas exigências impedem a espera circular

# Impedimento de deadlocks

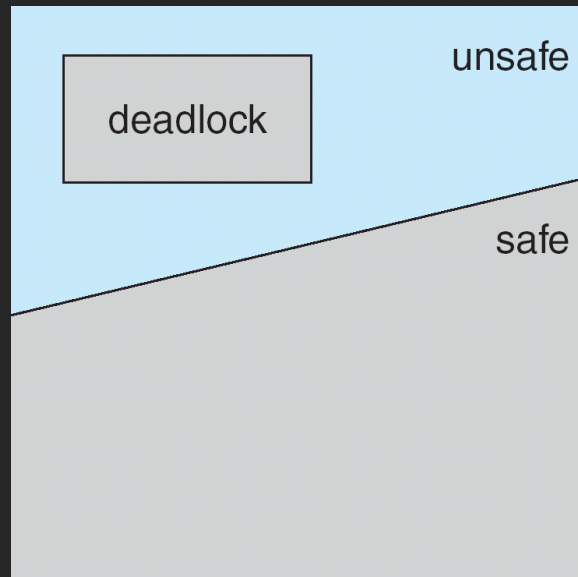
- Existem diversos algoritmos para impedimento de deadlocks:
  - Diferem em
    - Quantidade e tipo de informação requerida
  - Modelo mais simples e útil
    - Cada processo deve declarar o **número máximo** de recursos de cada tipo que irá usar
    - Algoritmo faz análise para cada processo e, de acordo com recursos do sistema, garante que jamais haverá deadlock

# Impedimento de deadlocks

- **Estado de segurança:** ocorre se o sistema puder alocar todos os recursos para todos os processos e impedir deadlock
- Definição formal: deve existir uma seqüência de segurança  $P_1, P_2, \dots, P_n$ 
  - Qualquer  $P_i$  pode solicitar mais recursos (até seu máximo) e obtê-los, considerando a utilização e possível liberação dos recursos por  $P_j$ , para todo  $j < i$ 
    - Nesse caso: se recursos de  $P_i$  não estiverem disponíveis, o processo deve aguardar até que  $P_j$  termine(m)
    - Quando  $P_i$  terminar,  $P_{i+1}$  poderá obter recursos e executar

# Impedimento de deadlocks

- Um estado de segurança não é um estado de deadlock
  - Um estado de deadlock é um estado de insegurança
  - Nem todos os estados de insegurança são de deadlock



# Impedimento de deadlocks

- Estado de insegurança pode levar a um estado de deadlock
  - Quanto mais seguro for o estado, mais SO poderá evitar insegurança (e deadlocks)
  - Em um estado de insegurança SO não se pode prevenir deadlocks
    - Dependerá do comportamento dos processos

# Impedimento de deadlocks

- Exemplo: sistema com 12 drives de fita e 3 processos P0, P1 e P2
  - P0 pode necessitar de 10 drives
  - P1 pode necessitar de 4 drives
  - P2 pode necessitar de 9 drives



# Impedimento de deadlocks

- Exemplo (cont.):
  - No tempo  $t_0$  há 3 drives de fita livres
  - O sistema está em estado de segurança
  - A seqüência de execução P1,P0,P2 satisfaz à condição de segurança

|    | Necessidades máximas | Em uso |
|----|----------------------|--------|
| P0 | 10                   | 5      |
| P1 | 4                    | 2      |
| P2 | 9                    | 2      |

# Impedimento de deadlocks

- Exemplo (cont.):
  - Sistema pode evoluir de estado de segurança para insegurança
    - Suponha que P2 solicite e obtenha mais 1 drive: estado de insegurança
      - Somente P1 poderá obter todos os seus drives
      - Quando P1 liberar, haverá 4 drives livres
      - P0 e P2 deverão esperar indefinidamente: deadlock
    - Caso P2 solicite, o sistema deverá fazer ele aguardar e manter uma seqüência de execução segura

# Impedimento de deadlocks

- A cada solicitação: SO deve decidir se processo será atendido ou se deverá aguardar (para impedir deadlock)
  - Mesmo havendo recursos livres, às vezes processo deverá aguardar (para garantir seqüência segura)
  - Menor utilização de recursos!

# Impedimento de deadlocks

- Algoritmo do grafo de alocação de recursos:
  - $P_i \rightarrow R_j$ : **aresta de solicitação** ( $P_i$  solicitou instância  $R_j$  e aguarda)
  - $R_j \rightarrow P_i$ : **aresta de atribuição** (instância  $R_j$  foi alocada a  $P_i$ )
  - Novo tipo de aresta
    - $P_i \rightarrow R_j$ : **aresta de requisição**
      - Indica que  $P_i$  poderá solicitar uma instância do recurso  $R_j$  no futuro
    - Aresta possui mesma direção que a de solicitação
      - Graficamente é representada por linha tracejada
    - Quando  $P_i$  solicitar  $R_j$ : a aresta de requisição  $P_i \rightarrow R_j$  torna-se uma aresta de solicitação
    - Quando um recurso  $R_j$  for liberado por  $P_i$ : a aresta de atribuição  $R_j \rightarrow P_i$  torna-se uma aresta de requisição  $P_i \rightarrow R_j$

# Impedimento de deadlocks

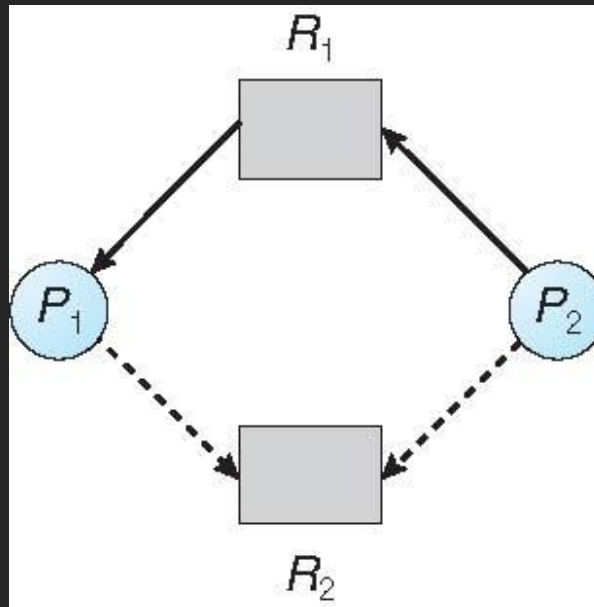
- Utilizando um grafo de alocação de recursos:
  - Todos os recursos que P irá utilizar deverão ser previamente requisitados
    - Antes de P iniciar execução: G deve conter todas arestas de requisição
- Funcionamento do algoritmo
  1. Construção do grafo de alocação de recursos
  2. Quando  $P_i$  solicita recurso  $R_j$ 
    - Solicitação será atendida somente se a conversão da **aresta de requisição**  $P_i \rightarrow R_j$  em **aresta de atribuição**  $R_j \rightarrow P_i$  não resultar em ciclo no grafo

# Impedimento de deadlocks

- Se houver ciclo: a alocação deixará o sistema em estado de insegurança (e poderá haver deadlock)
  - Quando uma solicitação de  $P$  gera um ciclo,  $P$  terá que aguardar para realizar novamente a solicitação e continuar a execução
- Detecção de ciclos em grafos:
  - Algoritmo tem complexidade de tempo  $O(n^2)$ , onde  $n$  é o número de processos

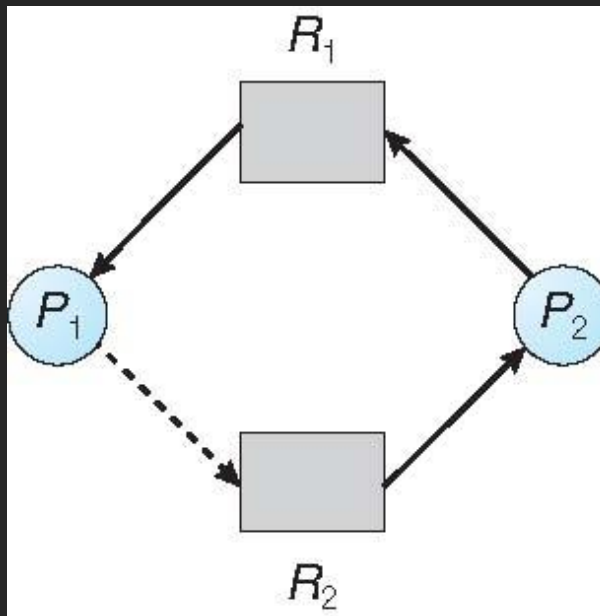
# Impedimento de deadlocks

- Exemplo:  $P_1$ ,  $P_2$  e  $R_1$ ,  $R_2$ 
  - Não há ciclo



# Impedimento de deadlocks

- Exemplo (cont.):
  - $P_2$  solicita  $R_2$  (recurso disponível!)
    - Forma ciclo:  $R_2$  não será alocado a  $P_2$





# Impedimento de deadlocks

- Grafo de alocação de recursos
  - Não é aplicável quando os recursos possuem múltiplas instâncias
- Algoritmo do banqueiro
  - Aplicável para múltiplas instâncias
  - Menos eficiente que grafo de alocação
  - Idéia: banco não deve alocar dinheiro disponível em caixa de forma que não possa satisfazer mais todos seus clientes

# Impedimento de deadlocks

- Funcionamento do algoritmo do banqueiro:
  - Quando novo processo entra no sistema:
    - Deve declarar o número máximo de instâncias de cada tipo de recurso que irá necessitar (não pode exceder o máximo do sistema)
  - Quando processo solicita conjunto de recursos:
    - Algoritmo determina se a alocação deixará o sistema em estado de segurança
      - Caso fique em segurança: aloca
      - Caso contrário: aguardar até que outros processos liberem o suficiente

# Impedimento de deadlocks

- Estruturas de dados:

- $n$  é o número de processos e  $m$  o número de tipo de recursos
- $\text{Disponível}[1..m]$ : quantidade de instâncias disponíveis de cada tipo
- $\text{Max}[1..n][1..m]$ : demanda máxima para cada processo.  $\text{Max}[i][j]$  é a quantidade máxima de instâncias que  $P_i$  pode solicitar de  $R_j$
- $\text{Alocação}[1..n][1..m]$ : quantidade de instâncias de recursos alocados a cada processo
- $\text{Necessidade}[1..n][1..m]$ : quantidade de recursos faltantes que cada processos pode necessitar para executar
  - $\text{Necessidade}[i][j] = \text{Max}[i][j] - \text{Alocação}[i][j]$
- Estruturas variam em valor e tamanho, de acordo com o tempo

# Impedimento de deadlocks

- Notação: dados dois vetores  $X$  e  $Y$  de tamanho  $n$ ,  $X \leq Y$  se  $X[i] \leq Y[i]$ , para todo  $i=1, \dots, n$ 
  - Ex.:  $X=\{1,7,3,2\} \geq Y=\{0,3,2,1\}$
  - $Y < X$  se  $Y \leq X$  e  $Y \neq X$
- Linha  $i$  das matrizes Alocação e Necessidade
  - Referenciadas como vetores Alocação[ $i$ ] e Necessidade[ $i$ ]

# Impedimento de deadlocks

- Algoritmo de segurança: verifica se sistema está ou não em estado de segurança
  1. Trabalho[1,...,m] := Disponível; Término[1,...,n] := falso
  2. Ache um  $i$  tal que:
    - Término[ $i$ ] = falso e Necessidade[ $i$ ]  $\leq$  Trabalho. Se não existir tal  $i$ , vá para o Passo 6;
  3. Trabalho := Trabalho + Alocação[ $i$ ];
  4. Término[ $i$ ] = true;
  5. Vá para o Passo 2;
  6. Se Término[ $i$ ] = true para todo  $i$ , então o sistema está em estado de segurança;
- Exige  $O(mn^2)$  operações

# Impedimento de deadlocks

- Seja Solicitação[1,...,n][1,...,m]: solicitações de recursos pelos processos
- Algoritmo de solicitação de recursos
  1. Se Solicitação[i]  $\leq$  Necessidade[i], vá para o Passo 2. Senão, emita erro;
  2. Se Solicitação[i]  $\leq$  Disponível, vá para o Passo 3. Senão,  $P_i$  deverá aguardar;
  3. Verifique a possibilidade de alocar recursos solicitados por  $P_i$  e atualize o estado das variáveis de alocação como segue:
    - Disponível := Disponível – Solicitação[i];
    - Alocação[i] := Alocação[i] + Solicitação[i];
    - Necessidade[i] := Necessidade[i] – Solicitação[i];

# Impedimento de deadlocks

- Se o estado de alocação de recursos resultante for seguro:  $P_i$  obtém recursos
- Caso contrário:
  - Estado das variáveis de alocação deve ser restaurado ao valor anterior
  - $P_i$  deverá aguardar pelas solicitações contidas em Solicitação[i]

# Detecção de deadlocks

- Se SO não possui sistema de prevenção ou impedimento, pode usar mecanismo de detecção de deadlocks
- Detecção de deadlocks
  - Algoritmo que verifica se há deadlock no sistema
    - Overhead em tempo de execução: como outros mecanismos
  - Algoritmo para recuperar sistema de deadlock
    - Overhead extra

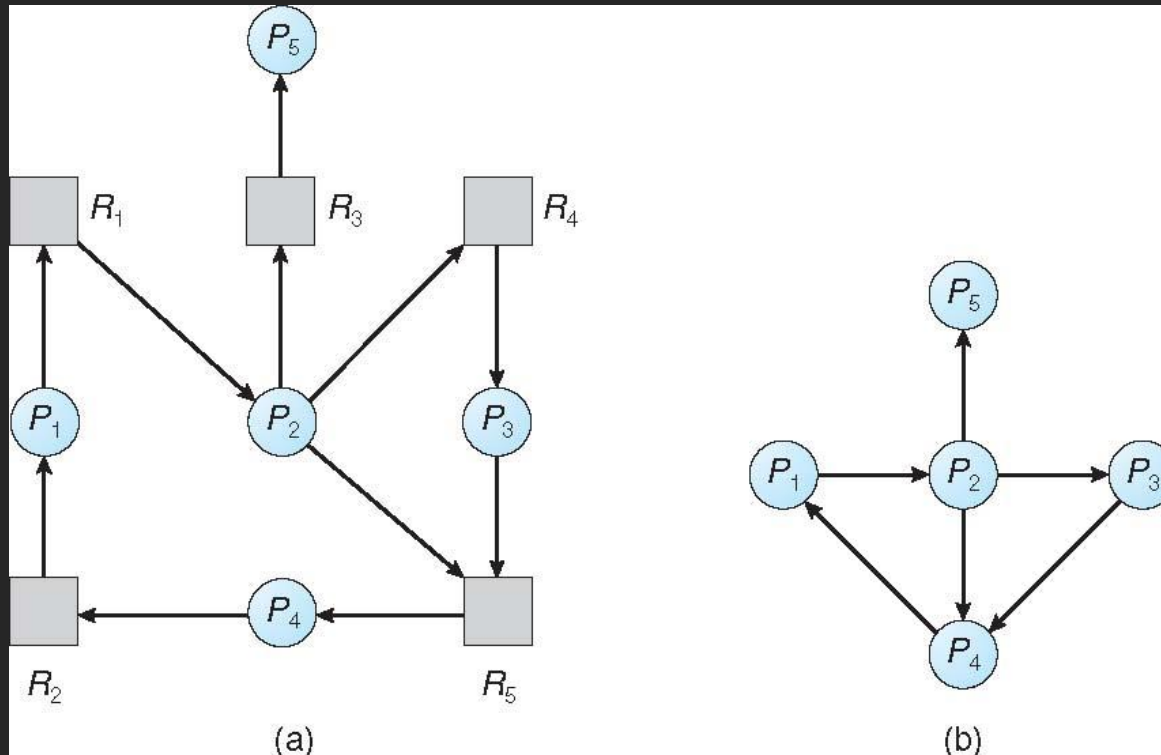


# Detecção de deadlocks

- Detecção com uma instância de recurso:
  - Usa grafo **espere-por**
    - Variante do grafo de alocação de recursos
      - Não há vértices indicando recursos, só processos
      - Aresta  $P_i \rightarrow P_j$  indica que  $P_i$  está esperando recurso que  $P_j$  está utilizando
    - Há deadlock se e somente se o grafo contiver um ciclo
  - Para detectar
    - SO deve construir e manter atualizado o grafo espere-por
    - Periodicamente: verificar a existência de ciclos

# Detecção de deadlocks

- Exemplo de grafo de alocação de recurso (a) e grafo espera-por correspondente (b):



# Detecção de deadlocks

- Detecção com diversas instâncias de recursos: algoritmo semelhante ao do banqueiro
  - Estruturas de dados: Disponível[1..m], Alocação[1..n][1..m] e Solicitação[1..n][1..m]. Trabalho[1..m], Término[1..n]
  - 1. Trabalho := Disponível; Término[i] := Alocação[i] ≠ 0 ? false : true;
  - 2. Ache índice i tal que
    - Término[i] = false e Solicitação[i] ≤ Trabalho
    - Se não houver i, vá para Passo 4
  - 3. Trabalho := Trabalho + Alocação[i]; Término[i] := true; Vá para o Passo 2;
  - 4. Se Término[i] = false para algum  $1 \leq i \leq n$ , então há deadlock. Cada processo i onde isso ocorre está em deadlock;

# Detecção de deadlocks

- Quando invocar algoritmo de detecção?
  - Fatores a considerar:
    - Qual a frequência esperada de ocorrência de um deadlock?
      - Diretamente proporcional à frequência: recursos ficam ociosos com deadlock
      - Deve-se atentar, porém, para o overhead...
    - Quantos processos serão afetados, se houver deadlock?
      - Número de processos em deadlock tende a crescer com o passar do tempo

# Recuperação de deadlocks

- Possibilidades para recuperação de deadlocks:
  - Detectar deadlock e informar operador
  - Sistema realizar recuperação automática
- Recuperação pode ser feita através de
  - Término de processos
    - Todos, um a um, ou até desfazer ciclo
      - Em qualquer caso: como recuperar um processo terminado (ex.: no meio de E/S)?
  - Preempção de recursos
    - Seleção da vítima: quais recursos podem sofrer preempção?
    - Recondução: o que fazer com o processo? como reiniciá-lo?
    - Inanição: algum processo sempre perderá?